

Structures conditionnelles

NSI 1ere – Octobre 2026

cours et fiche d'exercices sur les structures if.. elif.. else et les boucles non bornées while

1 Cours - Instructions conditionnelles et boucles non bornées

1.1 1. Instructions conditionnelles

1.1.1 1.1 Définition

Une *instruction conditionnelle* permet d'exécuter un bloc de code **seulement si** une condition est vraie :

```
1  if condition:
2      bloc_à_exécuter
```

La *condition* est une expression dont la valeur est un **booléen** : `True` (vrai) ou `False` (faux).

Exemple :

```
1  if prix_essence > 1.8:
2      print('Trop cher')
```

1.1.2 1.2 Écrire une condition : les opérateurs de comparaison

Opérateur	Signification	Exemple	Résultat
<code>==</code>	égal à	<code>5 == 5.0</code>	<code>True</code>
<code>!=</code>	différent de	<code>3 != 4</code>	<code>True</code>
<code><</code>	strictement inférieur	<code>7 < 7</code>	<code>False</code>
<code><=</code>	inférieur ou égal	<code>7 <= 7</code>	<code>True</code>
<code>></code>	strictement supérieur	<code>10 > 2</code>	<code>True</code>
<code>>=</code>	supérieur ou égal	<code>2 >= 10</code>	<code>False</code>
<code>in</code>	appartient à (opérateur d'appartenance)	<code>"a" in "banane"</code>	<code>True</code>

Attention : `=` sert à **affecter** une valeur à une variable (`x = 3`), alors que `==` sert à **comparer** deux valeurs (`x == 3`). Écrire `if x = 3:` provoque une erreur de syntaxe.

Astuce : on peut tester une condition directement dans le shell (la console) pour voir sa valeur :

```
1  >>> prix_essence = 1.95
2  >>> prix_essence > 1.8
3  True
```

1.1.3 1.3 Les blocs du programme et l'indentation

En Python, on utilise l'**indentation** (le retrait en début de ligne) pour délimiter les blocs de code.

- La ligne `if condition` se termine **toujours par deux-points** `:`.

- Toutes les lignes du bloc conditionnel sont décalées **du même retrait**. La convention en Python est d'utiliser **4 espaces** (la touche *TAB* de la plupart des éditeurs Python insère automatiquement 4 espaces).
- Il ne faut **jamais mélanger** tabulations et espaces dans un même programme.
- La première ligne qui revient à l'alignement du `if` marque la **fin du bloc**.

```

1  if prix_essence > 1.8:
2      print('Trop cher')           # dans le bloc : exécutée seulement si la
3      condition est vraie
4      print('Je prends le vélo')   # dans le bloc
print('Bonne journée')           # hors du bloc : toujours exécutée

```

Une erreur d'indentation provoque le message `IndentationError`.

1.1.4 1.4 L'alternative `if - else`

Une instruction `if - else` contient une instruction `if` dont le bloc s'exécute si la condition vaut `True`, et une clause `else` dont le bloc s'exécute si la condition vaut `False`.

`else` est ce que l'on appelle *l'alternative*. Il n'est **jamais suivi d'une condition**.

```

1  if hauteur_plant < 3:
2      print('laisser le plant dans la serre')
3  else:
4      print('le mettre dehors')

```

Remarque : observez bien l'alignement vertical qui distingue les instructions `if` et `else` (non indentées) des lignes exécutées de manière conditionnelle (indentées).

Dans une alternative, **un seul des deux blocs** est exécuté, jamais les deux.

1.1.5 1.5 Conditions multiples `if - elif - else`

Un bloc `if - elif - else` comprend une première instruction `if`, puis une suite de tests `elif` (contraction de *else if*, « sinon si ») effectués si les tests précédents ont échoué, puis un bloc `else` qui s'exécute si **tous** les tests ont échoué.

Même s'il n'est pas obligatoire, il est fortement recommandé de terminer une série de conditions `elif` par un bloc `else`.

```

1  if hauteur_plant < 3:
2      print('laisser le plant dans la serre')
3  elif hauteur_plant < 10:
4      print('le sortir le jour')
5  elif hauteur_plant < 15:
6      print('le mettre dehors')
7  else:
8      print('Prêt pour la récolte')

```

Point essentiel : les conditions sont testées dans l'ordre, de haut en bas. Dès qu'une condition est vraie, son bloc est exécuté, et les conditions suivantes ne sont pas examinées. Un seul bloc est donc exécuté.

Conséquence : dans l'exemple ci-dessus, la ligne `elif hauteur_plant < 10` n'est atteinte que si `hauteur_plant < 3` est faux. Elle correspond donc en réalité à la condition `3 <= hauteur_plant < 10`.

L'ordre des tests est donc important. Le programme suivant est faux :

```

1  if hauteur_plant < 15:
2      print('le mettre dehors')      # un plant de 1 cm serait mis dehors !
3  elif hauteur_plant < 3:
4      print('laisser le plant dans la serre')  # ce bloc n'est jamais exécuté

```

Rappelez-vous :

- `if` et `elif` sont **toujours suivis d'une condition** et de `:`
- `else` n'est **pas suivi d'une condition**, seulement de `:`. C'est l'alternative, dont le bloc est exécuté lorsqu'aucune des conditions précédentes n'est vraie.

1.1.6 1.6 Combiner des conditions : `and`, `or`, `not`

Les **opérateurs logiques** permettent de combiner plusieurs conditions :

Expression	Vaut <code>True</code> si...
<code>cond1 and cond2</code>	les deux conditions sont vraies
<code>cond1 or cond2</code>	au moins une des deux conditions est vraie
<code>not cond</code>	la condition est fausse

Table de vérité :

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Exemples :

```

1  if age >= 12 and age < 18:
2      print("Tarif adolescent")
3
4  if jour == "samedi" or jour == "dimanche":
5      print("C'est le week-end")

```

Remarques :

- Python accepte aussi l'écriture mathématique `12 <= age < 18`.
- Il faut répéter la variable de chaque côté : `jour == "samedi" or "dimanche"` est une **erreur** (cette condition est toujours vraie !).
- En cas de doute sur les priorités, utilisez des parenthèses : `(a > 0 and b > 0) or c == 0`.

1.1.7 1.7 Conditions imbriquées

On peut placer une instruction conditionnelle **à l'intérieur** du bloc d'une autre. Chaque niveau d'imbrication ajoute un retrait de 4 espaces.

Exemple : accès à un manège réservé aux personnes mesurant entre 1,20 m et 1,95 m

```

1  if taille >= 1.20:
2      if taille <= 1.95:
3          print("Accès autorisé")

```

```

4     else:
5         print("Trop grand pour ce manège")
6     else:
7         print("Trop petit pour ce manège")

```

Le message « Accès autorisé » s’affiche si `taille >= 1.20` et `taille <= 1.95`. Une condition imbriquée joue donc le même rôle qu’un `and`. On aurait pu écrire :

```

1     if taille >= 1.20 and taille <= 1.95:
2         print("Accès autorisé")

```

La version imbriquée est plus longue, mais elle permet d’afficher un message différent selon la raison du refus.

1.1.8 1.8 Tester la divisibilité : `//` et `%`

Pour deux entiers, Python calcule le quotient et le reste de la **division euclidienne** :

Opérateur	Rôle	Exemple	Résultat
<code>//</code>	quotient entier	<code>17 // 5</code>	<code>3</code>
<code>%</code>	reste (modulo)	<code>17 % 5</code>	<code>2</code>

car $17 = 5 \times 3 + 2$.

Un entier `n` est **divisible** par `d` si et seulement si le reste de la division de `n` par `d` est nul, c’est-à-dire si `n % d == 0`.

```

1     n = 35
2     if n % 5 == 0:
3         print("n est un multiple de 5")
4     else:
5         print("n n'est pas un multiple de 5")

```

Autres opérations utiles : la puissance s’écrit `**` (par exemple `3 ** 2` vaut `9`) et l’arrondi s’obtient avec `round(x, 2)` (deux chiffres après la virgule).

1.2 2. Condition sur une valeur entrée (`input`)

La fonction `input` est une **entrée** : elle affiche un message, attend que l’utilisateur tape une valeur et valide avec *Entrée*, puis renvoie cette valeur au programme.

La **valeur renvoyée par `input` est TOUJOURS de type `str`** (chaîne de caractères), même si l’utilisateur tape un nombre.

```

1     >>> reponse = input("Entrer un nombre : ")
2     Entrer un nombre : 12
3     >>> reponse
4     '12'
5     >>> type(reponse)
6     <class 'str'>

```

Pour faire des calculs ou des comparaisons numériques, il faut **convertir** la valeur saisie :

Fonction	Convertit en	Exemple	Résultat
<code>int(...)</code>	entier	<code>int("12")</code>	12
<code>float(...)</code>	nombre à virgule	<code>float("1.75")</code>	1.75
<code>str(...)</code>	chaîne de caractères	<code>str(12)</code>	'12'

Sans conversion, on obtient des résultats surprenants ou des erreurs :

```

1  >>> "3" + "4"           # concaténation de deux chaînes
2  '34'
3  >>> "12" > 9             # comparaison entre str et int impossible
4  TypeError: '>' not supported between instances of 'str' and 'int'
5  >>> int("1.75")          # un nombre à virgule ne peut pas être converti par int
6  ValueError: invalid literal for int() with base 10: '1.75'

```

On regroupe souvent la saisie et la conversion sur une seule ligne :

```

1  note = float(input("Entrer votre note sur 20 : "))
2  if note >= 16:
3      print("Mention très bien")
4  elif note >= 14:
5      print("Mention bien")
6  elif note >= 12:
7      print("Mention assez bien")
8  elif note >= 10:
9      print("Admis")
10 else:
11     print("Non admis")

```

Conseils :

- Utiliser `int` pour une valeur entière (un âge, un nombre d'objets), `float` pour une valeur décimale (une taille, une masse, un prix).
- Écrire un message clair pour que l'utilisateur sache quoi saisir, et dans quelle unité : "Entrer votre taille en mètres (ex : 1.75) : ".
- En Python, le séparateur décimal est le **point** : 1.75 et non 1,75.

1.3 3. Afficher un résultat : `print` et les f-strings

On peut afficher plusieurs éléments séparés par des virgules :

```

1  print("Vous avez", points, "points.")

```

Une **f-string** (chaîne précédée de la lettre `f`) permet d'insérer la valeur d'une variable ou d'une expression entre accolades `{ }` :

```

1  prix = 3.2
2  nombre = 3
3  print(f"Vous achetez {nombre} baguettes pour {prix} euros.")
4  print(f"Le double de {nombre} vaut {2 * nombre}.")
5  print(f"Prix arrondi : {prix:.2f} €") # 2 chiffres après la virgule : 3.20

```

1.4 4. Boucle non bornée : `while`

1.4.1 4.1 Syntaxe

Une boucle `while` (« tant que ») **répète** un bloc d'instructions **tant qu'une condition est vraie** :

```
1 while condition:
2     bloc_à_répéter
```

Fonctionnement :

1. La condition est évaluée.
2. Si elle vaut `True`, le bloc est exécuté (c'est une **itération**), puis on revient à l'étape 1.
3. Si elle vaut `False`, la boucle s'arrête et le programme continue après le bloc.

On parle de boucle **non bornée** car on ne connaît pas toujours à l'avance le nombre d'itérations. Si la condition est fausse dès le départ, le bloc n'est **jamais** exécuté.

1.4.2 4.2 Compteur et accumulateur

Deux types de variables reviennent très souvent dans les boucles :

- un **compteur** augmente de 1 à chaque itération : `n = n + 1` (ou en abrégé `n += 1`) ;
- un **accumulateur** additionne des valeurs au fil des itérations : `total = total + valeur`.

Ces variables doivent être **initialisées avant la boucle** (en général à `0`).

Exemple : une tirelire. On économise 15 € par semaine. Combien de semaines faut-il pour atteindre au moins 100 € ?

```
1 epargne = 0      # accumulateur
2 semaines = 0    # compteur
3 while epargne < 100:
4     epargne = epargne + 15
5     semaines = semaines + 1
6     print(f"Il faut {semaines} semaines pour économiser {epargne} €.")
```

1.4.3 4.3 Le tableau de suivi

Pour comprendre (ou vérifier) le déroulement d'une boucle, on construit un **tableau de suivi** : une ligne par test de la condition, avec la valeur des variables **avant** l'itération.

<code>semaines</code> avant itération	<code>epargne</code> avant itération	condition <code>epargne < 100</code>
0	0	<code>True</code>
1	15	<code>True</code>
2	30	<code>True</code>
3	45	<code>True</code>
4	60	<code>True</code>
5	75	<code>True</code>
6	90	<code>True</code>
7	105	<code>False</code> → fin de la boucle

Le programme affiche : `Il faut 7 semaines pour économiser 105 €.`

Remarque : la condition est testée **une fois de plus** que le nombre d'itérations (7 itérations, 8 tests). Le dernier test, qui vaut `False`, arrête la boucle.

1.4.4 4.4 Terminaison d'une boucle : le variant

Une boucle `while` peut ne **jamais s'arrêter** : c'est une **boucle infinie**. Cela arrive si la condition reste toujours vraie, par exemple si on oublie de modifier la variable testée.

```
1  compte = 5
2  while compte > 0:
3      print(compte)      # oubli de compte = compte - 1 : boucle infinie !
```

Pour interrompre un programme bloqué : CTRL + C, ou le bouton *stop / interrompre* de l'éditeur (carré noir dans un notebook).

Pour **prouver** qu'une boucle se termine, on cherche un **variant de boucle** : une quantité **entière, positive** tant que la boucle tourne, et qui **diminue strictement** à chaque itération. Une telle quantité ne peut pas diminuer indéfiniment, donc la boucle finit par s'arrêter.

Exemple : un compte à rebours

```
1  compte = 5
2  while compte > 0:
3      print(compte)
4      compte = compte - 1
5  print("Décollage !")
```

- Le variant est `compte` : c'est un entier, positif tant que la boucle tourne, qui diminue de 1 à chaque itération.
- La boucle s'arrête quand `compte` atteint 0.
- Le nombre d'itérations se déduit de la valeur initiale du variant : ici **5 itérations** (plus généralement, `compte` itérations si `compte` est un entier positif au départ).

Dans l'exemple de la tirelire, le variant est la somme restant à économiser `100 - epargne` : elle diminue de 15 à chaque itération.

Cas d'une boucle qui dépend d'une saisie :

```
1  reponse = ""
2  while reponse != "oui":
3      reponse = input("Voulez-vous arrêter ? (oui/non) ")
```

Cette boucle ne s'arrête que si l'utilisateur tape exactement `oui` : sa terminaison **n'est pas garantie**. Pour être sûr qu'une boucle se termine quelles que soient les saisies, on la contrôle par un **compteur** :

```
1  essais = 0
2  while essais < 3:
3      reponse = input("Mot de passe : ")
4      essais = essais + 1
```

Ici, quoi que tape l'utilisateur, `essais` augmente de 1 à chaque itération : le variant `3 - essais` diminue strictement, et la boucle s'arrête après exactement 3 itérations.

1.4.5 4.5 Une boucle contenant une instruction conditionnelle

Le bloc d'une boucle peut contenir une instruction `if`. Attention aux **deux niveaux d'indentation** :

```

1  n = 0
2  pairs = 0
3  while n < 4:
4      valeur = int(input("Entrer un entier : "))
5      n = n + 1
6      if valeur % 2 == 0:
7          pairs = pairs + 1
8  print(f"Vous avez saisi {pairs} nombre(s) pair(s) sur {n}.")

```

1.5 5. Rappel : écrire une fonction

Une **fonction** regroupe des instructions sous un nom, pour pouvoir les réutiliser. Elle reçoit des **paramètres** et peut **renvoyer** un résultat avec `return`.

```

1  def mention(note):
2      if note >= 16:
3          return "très bien"
4      elif note >= 14:
5          return "bien"
6      elif note >= 12:
7          return "assez bien"
8      else:
9          return "sans mention"
10
11 # programme principal
12 ma_note = float(input("Entrer votre note : "))
13 print(f"Mention : {mention(ma_note)}")

```

- `return` **renvoie** une valeur au programme qui a appelé la fonction, et **termine** immédiatement la fonction.
- `print` se contente d'**afficher** : la valeur n'est pas récupérable par le programme.
- On peut tester une fonction dans le shell : `>>> mention(15)` renvoie `'bien'`.

1.6 6. Erreurs fréquentes

Erreur	Exemple fautif	Correction
Oubli des deux-points	<code>if x > 3</code>	<code>if x > 3:</code>
<code>=</code> au lieu de <code>==</code>	<code>if x = 3:</code>	<code>if x == 3:</code>
Condition après <code>else</code>	<code>else x < 3:</code>	<code>else:</code>
<code>elif</code> sans condition	<code>elif:</code>	<code>elif x < 10:</code>
Mauvaise indentation	bloc non décalé	4 espaces devant chaque ligne du bloc
Oubli de conversion de <code>input</code>	<code>age = input(...)</code> puis <code>age > 18</code>	<code>age = int(input(...))</code>
Virgule décimale	<code>taille = 1,75</code>	<code>taille = 1.75</code>
Condition incomplète avec <code>or</code>	<code>if x == 1 or 2:</code>	<code>if x == 1 or x == 2:</code>
Boucle infinie	variable de la condition jamais modifiée	modifier le variant dans le bloc

2 Exercices

2.1 Ex 1 : Vrai ou faux ?

On donne `a = 7`, `b = 12` et `mot = "python"`. Sans ordinateur, donner la valeur de chaque expression, puis vérifier dans le shell.

1. `a < b`
2. `a == 7.0`
3. `a != b and b > 10`
4. `not (a > 5)`
5. `a > 10 or b > 10`
6. `"y" in mot`
7. `b // 5 et b % 5`
8. `a % 2 == 0`
9. `b % 3 == 0 and b % 4 == 0`

2.2 Ex 2 : Blocs et indentation

Recopier le script sur votre copie en corrigeant les erreurs d'indentation.

```

1  if chantilly == True:
2  print('gaufre à la chantilly')
3      elif nutella == True:
4  print('gaufre au nutella')
5  else:
6  print('gaufre au sucre')
```

1. Qu'affiche le programme corrigé si `chantilly = False` et `nutella = False` ?
2. Qu'affiche-t-il si `chantilly = True` et `nutella = True` ? Pourquoi une seule gaufre est-elle affichée ?
3. *Remarque* : comme `chantilly` est déjà un booléen, on peut écrire simplement `if chantilly:`. Réécrire le script avec cette simplification.

2.3 Ex 3 : Prédire l'affichage

On considère les deux programmes suivants.

Programme A

```

1  if x > 5:
2      print("A")
3  elif x > 10:
4      print("B")
5  else:
6      print("C")
7  print("fin")
```

Programme B

```

1  if x > 5:
2      print("A")
3  if x > 10:
4      print("B")
5  print("fin")
```

1. Recopier et compléter le tableau des affichages :

valeur de x	affichage du programme A	affichage du programme B
12		
7		
3		

2. Dans le programme A, le bloc `print("B")` peut-il être exécuté ? Expliquer.
3. Modifier le programme A pour qu'il affiche "B" lorsque `x > 10`, "A" lorsque `5 < x <= 10` et "C" sinon.

2.4 Ex 4 : input et types

1. Prédire le résultat de chaque instruction, puis vérifier dans le shell :
 - `"3" + "4"`
 - `int("3") + int("4")`
 - `float("2.5") * 2`
 - `type(input("Entrer un nombre : "))` (on tape 8)
2. Le programme suivant provoque une erreur lorsqu'on tape 20. Recopier le message d'erreur, expliquer sa cause, puis corriger le programme.

```

1  age = input("Quel âge avez-vous ? ")
2  if age >= 18:
3      print("Vous êtes majeur")
4  else:
5      print("Vous êtes mineur")

```

3. On veut demander la taille d'une personne en mètres. Faut-il utiliser `int` ou `float` ? Écrire la ligne de saisie avec un message clair pour l'utilisateur.

2.5 Ex 5 : Instruction conditionnelle avec cas multiples

1. Écrire l'instruction conditionnelle qui affiche la phrase « il va faire très froid / froid / bon / chaud / très chaud » selon la valeur de la variable `t` (la température), à partir des seuils suivants :
 - `t >= 30` : il va faire très chaud
 - `25 <= t < 30` : il va faire chaud
 - `15 <= t < 25` : il va faire bon
 - `5 <= t < 15` : il va faire froid
 - `t < 5` : il va faire très froid

Aide : grâce au fonctionnement de `elif` (paragraphe 1.5 du cours), il n'est pas nécessaire d'écrire les deux bornes de chaque intervalle, à condition de choisir le bon ordre pour les tests.

2. Tester votre programme avec les valeurs `t = 32`, `t = 30`, `t = 20`, `t = 5` et `t = -3`. Pourquoi est-il important de tester les valeurs 30 et 5 ?
3. Modifier le programme pour que la température soit saisie par l'utilisateur (`input`).
4. Écrire une fonction `meteo` qui **retourne** « très froid / froid / bon / chaud / très chaud » selon le paramètre `t`. Puis écrire le programme qui appelle la fonction pour afficher « il va faire ... ».

2.6 Ex 6 : Divisibilité et opérateurs logiques

Une année est **bissextile** si elle est divisible par 4 mais pas par 100, **ou** si elle est divisible par 400.

1. Traduire chaque morceau de la règle en condition Python, en utilisant l'opérateur `%` :
 - « `annee` est divisible par 4 » : ...
 - « `annee` n'est pas divisible par 100 » : ...

- « `annee` est divisible par 400 » : ...
2. Écrire un programme qui demande une année à l'utilisateur et affiche si elle est bissextile ou non.
 3. Tester avec `2024` (bissextile), `2026` (non), `1900` (non) et `2000` (bissextile).

2.7 Ex 7 : Conditions imbriquées

Un cinéma applique les tarifs suivants : les moins de 14 ans paient 5 € ; les autres paient 7 € s'ils sont étudiants, et 10 € sinon.

1. Compléter le programme suivant, qui utilise deux conditions imbriquées :

```

1  age = int(input("Quel est votre âge ? "))
2  etudiant = input("Êtes-vous étudiant ? (oui/non) ")
3  if age < 14:
4      prix = ...
5  else:
6      if ...:
7          prix = ...
8      else:
9          prix = ...
10 print(f"Vous payez {prix} €.")

```

2. Traduire en langage naturel la condition pour laquelle le prix est de 7 € : *si ... alors si ... alors ...*
3. Réécrire le programme **sans imbrication**, avec une structure `if - elif - else` et l'opérateur `and`.

2.8 Ex 8 : Instruction conditionnelle avec alternative

Un boulanger désire un programme qui demande à l'utilisateur le nombre de baguettes qu'il désire et qui calcule le prix total, sachant qu'une baguette coûte 1,10 €. Il y a une promotion si le client achète plus d'une baguette : la première coûte 1,10 € et les baguettes suivantes coûtent 5 centimes de moins chacune. Le programme demande d'entrer le nombre de baguettes et affiche le prix que l'utilisateur doit payer.

La structure du programme doit s'inspirer du script suivant :

```

1  nombre = input("Combien de baguettes désirez-vous ? ")
2  if ...:
3      ...
4  else:
5      ...
6  print("Vous avez à payer", prix, "euros.")

```

1. Sans ordinateur, calculer le prix à payer pour 1, 2 et 5 baguettes.
2. Exprimer le prix en fonction de `nombre` dans le cas où `nombre > 1`.
3. Recopier le script et le compléter. *Attention* : la première ligne doit elle aussi être corrigée (voir le paragraphe 2 du cours).
4. Tester le programme avec les valeurs de la question 1. Si l'affichage comporte beaucoup de décimales (par exemple `5.3000000000000001`), utiliser `round(prix, 2)` ou une f-string avec `{prix:.2f}`.

2.9 Ex 9 : Boucle `while` et compte à rebours

```

1  compte = 10
2  while compte > 0:
3      print(compte)
4      compte = compte - 2
5  print("Décollage !")

```

1. Recopier et compléter le tableau de suivi :

compte avant itération	condition <code>compte > 0</code>	affichage
10	<code>True</code>	10
8		
...		

2. Combien d'itérations ont lieu ? Quel est le variant de boucle ?
3. On remplace la condition par `compte != 0`. Le programme s'arrête-t-il ? Et si, en plus, on remplace `compte = 10` par `compte = 9` ? Expliquer (vous pouvez tester, en sachant interrompre le programme : voir paragraphe 4.4).
4. Quelle condition est la plus sûre : `compte > 0` ou `compte != 0` ?

2.10 Ex 10 : La tirelire

On reprend l'exemple du cours (paragraphe 4.2), mais on économise 20 € par semaine, avec un objectif de 150 €.

1. Modifier le programme en conséquence.
2. Construire le tableau de suivi avec les colonnes : `semaines` avant itération, `epargne` avant itération, condition `True/False`.
3. Combien de semaines faut-il ? Quelle somme a-t-on alors économisée ?
4. Que se passe-t-il si on écrit par erreur `epargne = epargne - 20` dans la boucle ? Expliquer à l'aide de la notion de variant.

2.11 Ex 11 : Boucle et condition : compter les notes

On veut saisir 5 notes et compter combien sont supérieures ou égales à 10, et combien sont inférieures à 10.

1. Compléter le script suivant :

```

1  n = 0
2  au_dessus = 0
3  en_dessous = 0
4  while n < ...:
5      note = float(input("Entrer une note : "))
6      n = ...
7      if ...:
8          au_dessus = ...
9      else:
10         en_dessous = ...
11  print(f"{au_dessus} note(s) >= 10 et {en_dessous} note(s) < 10")

```

2. Quel est le rôle de la variable `n` ? Est-ce un compteur ou un accumulateur ? Et `au_dessus` ?
3. On saisit les notes 12, 8, 15, 9, 10. Recopier et compléter le tableau de suivi :

n avant itération	au-dessus avant itération	en-dessous avant itération	condition True/False	note saisie
0	0	0	True	12
1				8
2				15
3				9
4				10
5				(pas de saisie)

4. Que vaut toujours `au-dessus + en-dessous` à la fin du programme ? Pourquoi ?
5. Ce programme se termine-t-il quelles que soient les notes saisies ? Justifier.
6. *Pour aller plus loin* : ajouter un accumulateur `total` pour afficher aussi la moyenne des 5 notes.

2.12 Ex 12 : Trouver l'erreur

Chacun des programmes suivants contient une erreur. L'identifier, expliquer ce qui se passe à l'exécution, puis corriger.

Programme 1

```

1 x = int(input("Entrer un nombre : "))
2 if x == 1 or 2:
3     print("x vaut 1 ou 2")

```

Programme 2

```

1 i = 0
2 while i < 5:
3     print(i)
4     i = i + 1

```

Programme 3

```

1 total = 0
2 while total < 50:
3     ajout = 10
4     print(total)

```

Programme 4

```

1 note = float(input("Note : "))
2 if note >= 10:
3     print("Admis")
4 else note < 10:
5     print("Non admis")

```